Demo prep: everything you need for tomorrow

Read sections 1 to 3 tonight, rehearse section 4 out loud twice, and keep section 9 open on your phone during the demo.

1. The honest state of things (read first)

- **Check Vercel first.** Open the Vercel project and confirm the newest Production deployment says Ready. The latest commits have not been seen building there yet. If the newest one failed, the previous deployment stays live, so the site still works, but the newest features would be missing.
- **New since the last version:** run the five extra SQL files too (clinic-audit-log.sql, clinic-booking-guard.sql, clinic-reminders.sql, clinic-plan-renewals.sql, clinic-gateway.sql). Without clinic-audit-log.sql the Audit log page shows a setup notice. Tests, CI and Lighthouse now run automatically on GitHub; you can mention them in the viva as proof that it works.
- **Run the three earlier SQL files in Supabase** (SQL Editor, one at a time): clinic-claims.sql, clinic-patient-profile.sql, clinic-realtime.sql. Without claims.sql the Claims pages show an empty or error state. Without patient-profile.sql, saving My profile fails. Without realtime.sql, only the live notification bell is affected.
- **logins.txt** is no longer tracked in the repo and is listed in .gitignore. It stays in git history, so change those demo passwords in Supabase when the demo is over.
- **Dark mode, Afrikaans and the new pages have not been seen rendered** by me. Click through each once tonight in a real browser.
- **Record the 60 to 90 second backup** tonight (section 8). Nobody can do that for you.

2. The 10-item alignment checklist, with what to do

- **1. Production URL loads signed out and in.** Open https://nextjs-dashoard-psi.vercel.app in a private window. Then sign in as admin.
- **2. README with live link, demo login, run steps.** Done: the README now starts with the six parts (what it does, live link and demo login, how to run, stack and why, limitations, three next steps) plus a smoke checklist. Variable names only, no values.
- **3. Login protects the dashboard.** Signed out, open /medi-clinic. You are sent to /clinic-login. This is done by middleware.ts, which checks the Supabase session before the page loads.
- **4. Entity 1 works (patients).** Create "Seed patient 1", rename it, delete it. Only the owner can delete.
- **5. Entity 2 linked by a foreign key (appointments).** appointments.patient_id references patients. The create form has a patient dropdown that lists only the patients your role may see.
- **6. RLS proven with a second user.** Sign in as the patient in a private window. You see only your own records. Be ready to explain this out loud (section 6).
- **7. Chart 1 states the question.** Now titled "Is the practice growing? New patients per month", with axis captions, bars from zero, and the line "If the last three bars fall, the owner holds off on a second consulting day."
- **8. Chart 2, a different shape.** Donut titled "What share of this month's appointments are no-shows?", with a so-what line underneath.
- **9. Proposal in the repo.** proposal.md now has a "Who it is for and the two questions" section. Say what changed since you wrote it: the project grew from one billing dashboard to three dashboards.
- **10. Smoke test ticked, no secrets, last commit deployed.** Tick the README smoke list by hand tonight on production. Confirm Ready in Vercel.

Also: DEMO.md now has five numbered lines (hook, problem, walk, decision, limits) and exactly three next steps.

3. How everything works (know this and the viva is easy)

The big picture

One Next.js app, one Supabase project. Supabase handles sign-in and stores the data. Next.js draws the pages on the server and talks to Supabase as the signed-in person. The database, not the screen, decides what each person may see.

- **Pages are server components.** They fetch the data on the server, then send finished HTML. Writes (create, edit, delete) are server actions: a form submits to a function that runs on the server.
- **Sign-in.** Supabase Auth issues a session cookie. middleware.ts runs on every dashboard route and redirects signed-out people to the login page.
- **Roles.** Each user has a role stored in app_metadata.clinic_role (owner, admin, front_desk, doctor, pharmacist, stock_manager, patient). app_metadata can only be changed by the server or admin, never by the user, so a person cannot promote themselves. getClinicAccess() in app/medi-clinic/lib/access.ts reads it and gates every page.
- **Two layers of protection.** Layer one: the page checks the role and hides or redirects. Layer two, which is the one that matters: row-level security (RLS) policies in Postgres. Even if someone forged a request, the database returns nothing for rows they may not see.

Family Clinic: the admin side (your main demo)

- **Overview (/medi-clinic).** Three metric cards (new patients, appointments this month, no-show share), the two charts, and tomorrow's booked appointments with phone numbers. Hover a bar or slice to see its statistics card.
- **Patients.** List, add, edit. Patient numbers are assigned by a database trigger. Identity fields are protected by a trigger so they cannot be silently rewritten. Delete is owner-only.
- **Appointments.** Create with a patient dropdown and a provider; change status (booked, done, no-show). Owner, admin and front desk can manage; doctors read their own.
- **Calendar.** Week view. One doctor gives a time grid with their shifts shaded; "All doctors" gives a card list per day with full names. Hover for details.
- **Shifts.** Monday to Friday by default, with a weekly grid per provider. Admin can set availability.
- **Payments.** The in-person payment register for front desk, and the Care Plan payments. Export CSV downloads it.
- **Claims board.** Patients submit claims; staff move them through the stages. The database only allows legal moves: submitted to in review or rejected, in review to approved or rejected, approved to paid. An illegal move is refused by the database function, not just hidden in the interface.
- **Members.** Owner and admin see every Care Plan member with plan and status filters, and export to CSV.
- **Other roles.** Doctors choose "which doctor are you" in a pop-up and see their own dashboard. Pharmacist and stock manager use the pharmacy workspace (prescriptions, inventory). Front desk goes to the payment register.
- **Audit log (owner and admin).** Every change to patients, appointments, claims and plans, with who and when, but never the personal values.
- **Stock alerts.** The pharmacy page flags low stock and batches expiring within 30 days.
- **Behind the scenes.** Daily jobs send appointment reminders and renew Care Plans; the database refuses double bookings; the site is installable on a phone.
- **Extras.** Dark and light mode (toggle or key T), English and Afrikaans (key L), a guided tour (the ? button), keyboard shortcuts (press ?), and a bell that fills with live notifications when an appointment, payment, member or claim changes (Supabase Realtime).

Medical Aid and the patient side (what you show as the patient)

- **Same login system.** The patient signs in with a patient account; the role is "patient". Their portal is /medi-clinic/portal, and the Medical Aid dashboard is /medical-aid.
- **Portal tabs.** Overview, My profile, Appointments, Prescriptions, Medical aid, Family Care Plans, Documents, Payments, Support.
- **My profile.** The patient edits their name, phone and details. This goes through a database function, update_my_profile, which only works for a patient and only on their own row, and validates the input.
- **Medical Aid dashboard tabs.** Overview, Plans, Cards, Payments, Claims, Documents.
- **Plans.** Essential (R199), Family Plus (R549), Premium (R899) a month.
- **Joining a plan.** The patient saves a card (brand, last four digits and expiry only), picks a plan, and chooses which card pays. The function enrol_in_care_plan records the membership and a demo payment. No real money moves, and the screen says so.
- **Progress and payment tracking.** A progress bar shows the plan period and payments made.

- **Claims.** The patient submits a claim (submit_claim) and watches a tracker move through the stages as staff update it. They see only their own claims.
- **Documents.** Uploads go to a private storage bucket with policies, so only that patient and authorised staff can read them.

Why the data stays private (the core answer)

- Every table has RLS switched on. A policy is a rule attached to the table, for example "a patient may read a row only if the row belongs to them" or "staff may read rows of their own clinic".
- The rule is evaluated by Postgres on every query, using the signed-in user's id and role from the token. The app cannot skip it.
- Sensitive actions (enrol, submit claim, change claim status, update profile) are database functions that check the role and the allowed moves themselves.

---

4. The 5-minute script (say the intent, not the clicks)

0:00 to 0:30 Hook

The Family Clinic Platform is a secure clinic system for family practices, where reception, doctors, pharmacy and patients each see only what their role allows. I built it with Next.js and Supabase, and it is live right now.

On screen: the landing page with the three dashboard cards.

0:30 to 1:00 Problem

Without a system like this, bookings, patient records, payments and medical aid claims sit in separate tools and shared spreadsheets. That is slow for families, and risky for health data, because a receptionist, a doctor and a pharmacist should not all see the same things.

1:00 to 3:30 Live walk (one happy path)

Sign in as admin.

Signing in puts me on the clinic overview. Notice I only see staff tools, because my role is admin.

Create. Open Appointments and book one, choosing a patient from the dropdown.

Every appointment is linked to a real patient, so I choose from a list. I cannot book for someone who does not exist, and the dropdown only shows patients I am allowed to see.

Edit. Change its status or time.

Reception can correct a booking in seconds, and the change is saved straight to the database.

Chart. Back on the overview, hover the bar chart, then the donut.

The owner asks two questions here. Is the practice growing? If the last three bars fall, they hold off on a second consulting day. And what share of appointments are no-shows? Here it is 4 of 46, and if that stays high, reception sends reminders the day before.

Medical side. Open the Claims board and Members briefly, then sign out.

The same data feeds the staff view of Care Plans and claims, and I can export it to a spreadsheet.

Sign in as the patient. Open My profile and change the phone number, then show Medical Aid and Claims.

A patient sees only their own appointments, plan and claims. Here they can edit their own phone number, join a plan and track a claim.

Sign out. Press T for dark mode, L for Afrikaans, then sign out.

3:30 to 4:15 One real decision

I chose to enforce access in the database with row-level security, and to store roles in settings users cannot edit. Hiding buttons is not enough: someone could tamper with the page. With row-level security, even a forged request gets nothing back, because the database itself refuses to show another patient's records.

4:15 to 5:00 Limits and next steps

Two honest limits. The payment is a labelled demo record: there is no gateway and no money moves. And I make no POPIA or compliance claim, so a legal and privacy review comes before real patient data. Next, I would connect a real payment gateway with verified webhooks, add refunds plus e-mail and SMS alerts, and complete the privacy review.

Timing tip: the walk is the part that overruns. If you are past 3:30 while still walking, skip the Members page. Aim to finish between 4:30 and 5:30.

5. What the panel scores, and how you hit it

- **It works.** Sign in, create, edit, chart, sign out, all on the production URL, in one smooth pass. Do not apologise for or explain around anything.
- **The pitch.** Hook in one sentence, problem in one sentence. Name the user: the owner and front desk of a small family clinic.
- **How it works.** The decision section: RLS, trusted roles, database functions. Have one sentence ready on the tech stack.
- **Honesty and next steps.** Name two real limits and three concrete next steps. Do not claim compliance.
- **Delivery and timing.** Speak to the panel, narrate why you are doing something, not "now I click". Finish between 4:30 and 5:30.

6. Likely viva questions, with answers to put in your own words

How does a user's data stay private from other users?

- Every table has row-level security. Each policy says who may read or write a row, for example a patient only their own rows and staff only their own clinic's rows.
- Postgres checks those rules on every query using the signed-in user's id and role, so the app cannot bypass them.
- Roles live in app_metadata, which users cannot edit.
- Proof: sign in as a second user and their lists are empty or only show their own records.

What was the hardest bug?

Pick one you truly lived through and say it in your own words. Two good ones:

- **Infinite recursion in RLS.** The patients policy checked prescriptions, and the prescriptions policy checked patients, so Postgres looped and errored. I fixed it with a security-definer helper function (pharmacist_can_access_patient) that does the check without re-triggering the policy. The fix is in fix-patients-rls-recursion.sql.
- **The failed Vercel build.** I added hover cards and exported a helper from a client file that server components called. Next.js does not allow that, so the build failed on Vercel, and the old version stayed live. I moved the helper into its own plain file (chart-tip.ts) and the build passed.

Why Supabase and RLS instead of checking in code?

Checks in the page can be forgotten or bypassed. A rule on the table protects the data everywhere, including any future page.

How do roles work?

The role is stored in the user's app_metadata, which only the server can set. The page reads it to decide what to show, and the database policies read the same role to decide what to return.

Is the payment real?

No. It is a labelled demo record. Only the brand, last four digits and expiry are stored, never the full card number. A real gateway with verified webhooks is my first next step.

How do you know it works?

Every push runs automated checks on GitHub: the security rules are tested on a real Postgres with a second user, the site is built and type-checked, browser tests visit the live site, and Lighthouse scores it for

accessibility.

What would you change if you had another week?

The three next steps: a real payment gateway, refunds with e-mail and SMS alerts, and the legal and privacy review.

What changed since your proposal?

It started as one billing dashboard. It grew into three dashboards on one login system: the clinic, the original invoice dashboard, and Medical Aid with Care Plans and claims.

Why Next.js and server actions?

Pages fetch data on the server next to the page, so there is little client code, and forms call server functions directly, which keeps secrets off the browser.

7. Tonight's checklist

- Confirm the newest Vercel deployment is Ready.
- Run clinic-claims.sql, clinic-patient-profile.sql and clinic-realtime.sql in Supabase.
- Sign in as admin and as the patient on production. Walk the whole script once with a timer.
- Click through dark mode and Afrikaans on the overview, calendar, claims and the patient portal.
- Tick the README smoke list.
- Record the backup (section 8).
- Charge your laptop, close other tabs, and set the browser zoom so text is readable from a distance.
- Have two windows ready: one normal for admin, one private for the patient.

8. The 60 to 90 second backup recording

- Record the screen with sound (Windows: Win+G or Xbox Game Bar, Mac: Shift+Cmd+5).
- Show only the happy path: sign in as admin, book an appointment, hover the two charts, sign out. Add ten seconds as the patient on My profile if there is time.
- Say one line per step, for example "Signing in as admin", "Booking an appointment for a real patient", "The owner's two questions", "Signing out".
- Save it on your laptop and on your phone, so you can play it if the internet fails.

9. If something goes wrong

- **Site will not load.** Say "Here is the recording of the same walk," and play the backup. Carry on with the decision and limits.
- **Sign-in fails.** Try the other account. Check you are on the production URL, not localhost.
- **A page errors.** Go back with the arrows (the back button is in every header) and move on. Do not debug live. Say "This is one of the limits I would fix next," only if it is true.
- **Claims or profile errors.** That means a SQL file has not been run. Skip it, and show the rest.
- **Out of time.** Cut the Members page and the dark mode and language moment. Never cut the decision or the limits.
- **Blank mind on a viva question.** Say "Let me think for a second," then use: who is signed in, which role, which policy, what the database returns.